

Ctrl-Alt-DefeatTheMarket

An Idiot's Guide to IMC Prosperity Algorithmic Trading

IMC Prosperity is about trading, and in practice it is split into two parts:

1. **Manual trading**
2. **Algorithmic trading**

The two are usually linked. Ideas from the manual trading side can often help with the algorithmic side, because the manual questions tend to highlight useful reflections, patterns, and themes.

That said, this guide is mainly about **algorithmic trading** — the practical side.

That is the part I care most about, and the part I want to focus on here.

I do respect that many people will want to work through the manual trading section as well, and there is definitely value in doing so. This just is not what this guide is centred around.

What this guide covers

- How do I get set up?
- What is actually going on?
- Where do I find resources?
- What is the system like?
- My own secret stash

A note on expectations

For now, most of us are assuming there will be **five rounds**, not including the tutorial round, just like in previous years.

We also expect the challenge to contain themes similar to earlier editions, such as:

- stationary assets
- drifting assets
- baskets
- derivatives
- macroeconomic impacts

How do I set up my first trader?

If you are new to the IMC Prosperity Challenge, this is the page you will usually land on first.

After signing up and logging in, click **Continue** to move to the initial setup.

Landing page

Figure 1: Landing page

Continue screen

Figure 2: Continue screen

You will then see a countdown timer. When available, click **Start Mission** to enter the challenge interface.

From here, you have two main options:

- click **Open Challenge** to upload your trading strategy
- scroll down to download the data capsule for products such as **Emeralds** and **Tomatoes**

The data capsule is useful for research, idea generation, and building your first trading strategy.

Clicking **Open Challenge** takes you to the upload page, where you can submit your trading file. Your submission must be a Python file ending in `.py`.

After uploading a trader, you will see performance output such as graphs and results. You can also compare multiple uploads to see which strategy performed best.

You can also open the side menu for more resources. I strongly recommend reading the **Wiki** before writing your first strategy.

The **Wiki** contains important information about each round, the products, and the overall challenge structure. Read it carefully before you start coding.

What should I use to code my trader?

A good place to start is **Visual Studio Code**.

I personally use **Spyder** through **Anaconda**, which also works well.

- Visual Studio Code
- Anaconda Navigator

What code do we get from IMC to start with?

Below is the basic starter template provided by IMC:

Start mission

Figure 3: Start mission

Upload trader

Figure 4: Upload trader

Results graph

Figure 5: Results graph

```
from datamodel import OrderDepth, UserId, TradingState, Order
from typing import List
import string

class Trader:

    def bid(self):
        return 15

    def run(self, state: TradingState):
        """Only method required. It takes all buy and sell orders for all
        symbols as an input, and outputs a list of orders to be sent."""

        print("traderData: " + state.traderData)
        print("Observations: " + str(state.observations))

        # Orders to be placed on exchange matching engine
        result = {}
        for product in state.order_depths:
            order_depth: OrderDepth = state.order_depths[product]
            orders: List[Order] = []
            acceptable_price = 10 # Participant should calculate this value
            print("Acceptable price : " + str(acceptable_price))
            print(
                "Buy Order depth : " + str(len(order_depth.buy_orders)) +
                ", Sell order depth : " + str(len(order_depth.sell_orders))
            )

            if len(order_depth.sell_orders) != 0:
                best_ask, best_ask_amount = list(order_depth.sell_orders.items())[0]
                if int(best_ask) < acceptable_price:
                    print("BUY", str(-best_ask_amount) + "x", best_ask)
                    orders.append(Order(product, best_ask, -best_ask_amount))
```

Side menu

Figure 6: Side menu

Wiki page

Figure 7: Wiki page

Screenshot

Figure 8: Screenshot

```
if len(order_depth.buy_orders) != 0:
    best_bid, best_bid_amount = list(order_depth.buy_orders.items())[0]
    if int(best_bid) > acceptable_price:
        print("SELL", str(best_bid_amount) + "x", best_bid)
        orders.append(Order(product, best_bid, -best_bid_amount))

result[product] = orders

# String value holding Trader state data required.
# It will be delivered as TradingState.traderData on next execution.
traderData = "SAMPLE"

# Sample conversion request. Check more details below.
conversions = 1
return result, conversions, traderData
```

What the fuck is alpha?

What is actually going on here?

In the tutorial round, we are given two assets: **Tomatoes** and **Emeralds**. At first glance, the distinction may seem to be simply “edible” versus “inedible”. In practice, though, they behave very differently in market terms.

If you open the data capsules and start exploring the files, you can begin to reverse-engineer the basic mechanics IMC has built into these products.

If you are not sure where to find the data, it is available here: Tutorial folder

Before going through each asset in detail, it is worth keeping one thing in mind: the challenge consists of **5 rounds**, and each round usually introduces a new mechanism, often through one or more additional assets.

For each product, the first step is usually the same: download the data capsule, extract the `.csv` files, and load them into a notebook or analysis environment of your choice. Use Python, R, Excel, or even Power BI if you must.

It is also worth mentioning that **Jasper Merle’s backtester** was a major help during IMC Prosperity 3. You should not expect the results to match the live challenge one-to-one, but strong backtester performance is generally still a good

Screenshot

Figure 9: Screenshot

Screenshot

Figure 10: Screenshot

sign. For example, a backtest result around **35K** roughly translated to about **9K** in Round 1 of IMC 3, which was approximately **top 10%**.

- jmerle’s backtester
- jmerle’s optimizer

Tutorial round assets

Emeralds, much like **Rainforest Resin** in IMC 3, behave like a classic stationary asset.

If you plot the historical data for Emeralds, you will see that the mid-price stays centered around **10,000** and oscillates around that level with a spread of roughly **16**. That makes it a strong candidate for a straightforward market-making strategy.

Tomatoes, much like **Kelp** in IMC 3, show drift.

Because of that, you cannot simply deploy a static market-making strategy and call it a day. You need an approach that either adapts to the drift or actively benefits from it. That could include ideas such as drift-aware market making, short-horizon trend following, short-selling where appropriate, or other models that react to directional behaviour in the data.

Round 1

At the time of writing, Round 1 has not started yet, so we do not know what new products or mechanics it will introduce.

Homework I Did So You Don’t Have To

I originally started by dumping in a pile of old repos and useful resources.

Then I removed them.

Instead, I want to structure this repo around the core components that actually matter.

At its heart, IMC Prosperity is a challenge in **high-frequency trading**, **algorithmic trading**, and **market making**.

That means we should think beyond just “finding alpha” and consider the full system behind a trading strategy.

A useful way to break that system down is into five parts:

1. **Alpha engine**
2. **Risk engine**
3. **Inventory engine**
4. **Execution engine**
5. **Portfolio management**

Below is a more detailed view of each.

Alpha

A simple way to think about many IMC rounds is as a sequence of increasingly difficult pricing and prediction problems.

For each asset, IMC is effectively presenting a different type of alpha opportunity.

For a **stationary asset**, the core idea is simple: **buy low, sell high**.

This is the most basic form of trading logic. The price moves around a stable level, so the job is to identify when it is temporarily cheap or expensive relative to that anchor.

For an asset with **drift**, things get more interesting.

Now the goal is no longer just “buy low, sell high” around a fixed centre. Instead, you want to identify **local minima**, **local maxima**, and the direction of movement. In other words, the problem becomes one of slope, trend, and timing.

That is the beginning of **trend-following**.

As the rounds progress, IMC usually increases the complexity in the same general direction.

Imagine, for example, that Round 2 introduces an asset correlated with the drifting one. Now you are not just dealing with one moving curve, but with a relationship between multiple assets. That turns the problem into one of relative value and joint movement.

That is the beginning of **pairs trading**.

Later rounds may introduce **basket/ETF arbitrage**, **derivatives**, or more complex combinations of assets. These are really just further extensions of the same question:

What is mispriced, relative to what, and why?

As the products become more complex, the methods used to extract alpha become more complex too.

Please sir, can I have the Sauce?

Risk

If you think you can ignore risk, skip this section and enjoy the leaderboard consequences.

For the rest of us, risk is one of the central problems in trading.

It is not enough to have a profitable signal. You also need to control how much you can lose when the signal is wrong, late, noisy, or overwhelmed by changing market conditions.

That means thinking about:

- current exposure
- unrealised loss
- adverse price movement
- volatility
- concentration of positions

Sometimes risk mitigation means reducing size.

Sometimes it means exiting part of a position.

Sometimes it means closing the trade entirely.

A strategy without risk controls is not really a strategy. It is just an opinion with leverage.

Linky Linky Boom Boom

Inventory

You also need to know what you actually hold.

That is where inventory management comes in.

In the tutorial round, the maximum inventory for **Emeralds** and **Tomatoes** is **80**.

That does **not** mean you should trade right up to the edge all the time. It is usually a much better idea to introduce **soft limits** or even multiple layers of limits before you ever hit the hard cap.

One important thing to remember is that your orders are not magically separated by “idea” once they hit the market.

For example:

buy 10 EMERALDS @ 100

buy 15 EMERALDS @ 100

is effectively just:

buy 25 EMERALDS @ 100

The market does not care whether one order came from your alpha model and the other came from your inventory logic. They combine into the same net exposure.

This is why alpha, risk, and inventory cannot be designed in isolation. They must work together, otherwise one component may accidentally amplify or cancel another.

Useful Nonsense

Execution

Execution is where everything comes together.

Your alpha may be good. Your risk logic may be sensible. Your inventory controls may be clean.

But if your execution is poor, the whole system still underperforms.

Execution is the process of deciding how orders should actually be placed, sized, split, updated, cancelled, or held back.

That includes questions like:

Should you cross the spread or quote passively? Should you place one large order or several smaller ones? Should you cancel an order when conditions change? Should you stagger execution across time?

Here are two useful ways to think about it.

Hypothetical 1: cancelling bad fills Imagine you are not trading against a simulation, but against a brilliant opponent who always knows the future.

The only advantage you have is speed: you can cancel your orders before they are hit.

You post:

buy 10 EMERALDS @ 100

Then you notice that your opponent is about to sell into it.

Do you keep the order live, or cancel it?

Sometimes the answer is to keep it. But often, having a cancellation rule ready is the difference between being a liquidity provider and being someone else's exit liquidity.

Hypothetical 2: distributing a large order Now imagine your best alpha signal fires.

It is strong, but not perfect. Maybe it is right **80%** of the time, and sometimes it fires slightly too early or slightly too late.

Rather than sending one large order immediately, you might distribute the trade over time.

For example, instead of buying **100 EMERALDS** all at once, you could split that into **10 smaller orders** over the next **10 time steps**.

That kind of execution logic can reduce the damage from early entries, poor timing, or short-term noise.

Call it **distribution, slicing, or VWAP-style execution**. The exact method matters less than the principle:

good execution helps imperfect signals survive reality.

Definitely Not Insider Info

Portfolio management

This one sounds fancier than it needs to.

Portfolio management here does **not** mean long-term investing. It simply means deciding how to allocate capital, risk, and inventory across multiple strategies or assets.

For example:

- you may have two alpha signals on the same asset
- you may have one market-making signal and one trend signal
- you may have multiple related assets competing for limited inventory

At that point, you need some way to decide how much weight each idea should receive.

You could just go with: “I like this one more.”

That is a valid human strategy. It is also a good way to let more systematic people outrank you.

A better starting point is to think in terms of expected return, variance, correlation, and capital allocation.

Weapons of Mass Instruction

Techbros, unite!

On a more serious note:

For those who want to go deeper into the challenge, it helps to understand what the system actually looks like so we can approach it as if it were a simplified version of a real market.

As with any trading simulation, a few things are worth keeping in mind:

1. We are the trader, and we are competing against other traders — in this case, bots.
2. There is a matching engine, much like on a real exchange.
3. From the exchange data and messages, you can reconstruct the order book. At the moment, it is only 3 levels deep.
4. This is also where it stops being fully realistic: your orders do not appear to affect the bots beyond the decisions they are already making. There is no psychological component or adaptive market behaviour on the other side.

The dataset provided by the exchange contains only price and volume. That means you cannot identify who is placing which orders or build strategies around specific counterparties.

Everything beyond that is essentially your own trading shop setup.

We have effectively unlimited capital, but performance is measured by PnL, which is at least familiar territory. Given that, it makes sense to approach the challenge as scientifically and systematically as possible.

A few technical principles are worth considering:

1. Keep memory usage and computation time as efficient as possible.
2. Maintain strong internal control over inventory, risk, current PnL, exposures, executions, forecasting, signal quality, and anything else that matters to your process.
3. Build something generalisable. Yes, you *can* hand-tailor strategies for every tiny scenario and hard-code edge cases all over the place, and some teams did exactly that last year. But for most people, that is not the best lesson to take away from this.

Try not to build nonsense.

You are not coding tomorrow's market prices in advance. A better approach is to build a general framework that actually makes sense and could still be useful outside the competition. Use sensible tools: estimate mid-prices, fit simple models, test ideas properly.

Use something like a linear regression for mid-price prediction.

Do not write: “at time $t = 10$, mid-price = 100” “at time $t = 101$, mid-price = 103.5”

That is not modelling. That is just hard-coded fantasy.

Just fucking spoon-feed me, bro!

So, you've reached the end of your rope and want to know what to write in your trader without actually doing the work.

Well, I'm not going to do that until the round is over.

What I *will* do is keep giving you the same model until you're properly confused.

Markets

Let $\mathbf{A}$ be a market with assets:

- $\mathbf{A}$
- $\mathbf{A}$
- $\mathbf{A}$
- ...
- $\mathbf{A}$

Each asset may have different properties and behave differently.

We also allow derivatives and related structures:

- Let $\mathbf{A}^*$ be a derivative of $\mathbf{A}$, where $\mathbf{A}$ is the underlying asset.
 - This relationship may be repeated and does not need to be unique to $\mathbf{A}$.
 - Similarly, let $\mathbf{A}^*$ be a basket containing $\mathbf{A}$ and some other asset $\mathbf{A}$.
 - This construction may also be repeated and does not need to be strict to $\mathbf{A}$.
-

Traders

Let $\mathbf{B}$ be a group of traders operating on an exchange $\mathbf{Alpha}$, associated with the market $\mathbf{A}$.

The exchange functions as a matching engine for the traders in $\mathbf{B}$.

The objective of each trader $\mathbf{b} \in \mathbf{B}$ is to optimize the following dynamic:

- Let **Profit** or **Reward** be a metric assigned to trader $\mathbf{b}$.
- Each trader seeks to maximize this value as much as possible.
- Likewise, let **Loss** be another metric assigned to trader $\mathbf{b}$.
- Each trader seeks to minimize this value, pushing it as close to $\mathbf{0}$ as possible.

You already understand that this is essentially **PnL**.

But you also need to account for something more important:

Your profit and loss functions should not be optimized for a single best outcome. They should be optimized for **expected value**.

Because we are not one-trick ponies.

One or two good trades can turn a pony into a unicorn.

But capturing *all* the good trades turns the pony into a **legendary quantitative unicorn**.

Dynamics

We now understand the universe in a broad and admittedly superficial way.

The next goal is to understand the rules governing:

- the market,
- the assets,
- and the competition.

Because we do not only want to profit.

We want to profit **the most**.

This becomes a game of understanding how each asset behaves, and then capturing the safest and most repeatable profits possible.

That is how we uncover **all** the good trades.

Reducing losses — or even reducing the *possibility* of losses — is even better.

So what we really want is the method that yields:

- the **highest expected reward**, and
- the **lowest expected loss**.

So to represent a generalistic model of where prices are moving could be useful. These dynamics may be aggregate in nature, or they may appear as components within a broader model.

For example:

mid_price[t+1]
= immediate order-book matching at t
+ order-flow imbalance at t
+ drift / trend at t
+ mean reversion at t
+ diffusion / noise at t
+ volatility dynamics at t
+ liquidity / market-impact effects at t
+ jump or shock components at t
+ regime behaviour at t
+ cross-asset or exogenous drivers at t

One could also map it like this

mid_price[t+1]
= microstructure effect
+ directional component
+ corrective component
+ stochastic component

- + volatility component
- + liquidity / impact component
- + jump component
- + regime component
- + exogenous component

But here you are on a really generalistic model for market prices.

We then have to figure out how to earn from this model. Oh! But wouldn't you notice, could we just adjust our inventory from this?

Could we map out which asset would probably have the highest probability of a positive return?

Could this model be used on any asset?

Do you want to figure it out?

Btw here is a simple 1K profit .py file for your trader, but only on EMERALDS. Try experimenting

Inspirational links.

Jasper(all honors to him for the list) decided to share his list. So I guess we are simply bundling it up and sharing for everyone

Prosperity 1:

2nd place: <https://github.com/ShubhamAnandJain/IMC-Prosperity-2023-Stanford-Cardinal>

57th place: https://github.com/MichalOkon/imc_prosperity

91st place: https://github.com/nicolassinott/IMC_Proprosperity

322nd place: <https://github.com/jmerle/imc-prosperity>

Prosperity 2:

2nd place: <https://github.com/ericcccsliu/imc-prosperity-2>

9th place: <https://github.com/jmerle/imc-prosperity-2>

13th place: <https://github.com/pe049395/IMC-Prosperity-2024>

124th place: <https://github.com/cpartridge18/prosperity-24>

332nd place: <https://github.com/edmund870/2024-IMC-Global-Trading-Challenge>

368th place: <https://github.com/AcreixYuan/IMC-Prosperity-2>

381st place: <https://github.com/davidteather/imc-prosperity-2024> 489th place:

<https://github.com/stephen-w-choo/imc-prosperity-2024>

Prosperity 3:

2nd place: <https://github.com/TimoDiehm/imc-prosperity-3>

7th place: <https://github.com/chrispyroberts/imc-prosperity-3>

9th place: <https://github.com/CarterT27/imc-prosperity-3>

10th place: <https://github.com/YBansal95/imc-prosperity-3>

19th place: <https://github.com/JackRao123/prosperity-crushers>

24th place: <https://github.com/awatatani/imc-prosperity3-trading>

25th place: <https://github.com/jmerle/imc-prosperity-3>

29th place: <https://github.com/musashi-island/prosperity3>
44th place: <https://github.com/angus4718/imc-prosperity-3-public>
60th place: <https://github.com/FumeiYoruu/imc-prosperity-3>
89th place: <https://github.com/ZainAlSaffi-Dev/imc-prosperity3>
109th place: <https://github.com/Sylvain-Topeza/imc-prosperity-3>
131st place: <https://github.com/JamesCole809/IMC-Prosperity-3>
141st place: https://github.com/Akezh/imc_trading_2025
168th place: <https://github.com/aahiill/IMC-Prosperity-3>
172nd place: <https://github.com/KengLL/Prosperity-3-Neko> (2nd in manual)
238th place: https://github.com/milesmitchell/imc_prosperity_3
287th place: <https://github.com/JohnnyTung123/imc-prosperity-3>

Optiver Realized volatility prediction: 1st place: <https://www.kaggle.com/competitions/optiver-realized-volatility-prediction/writeups/nyanp-1st-place-solution-nearest-neighbors>

3rd place: <https://www.kaggle.com/competitions/optiver-realized-volatility-prediction/writeups/pksha-life-is-volatile-tentative-3rd-place-solution>

xth place: <https://github.com/taher-software/Optiver-Realized-Volatility-Prediction/tree/master>

Optiver Trading at the close:

1st place: <https://www.kaggle.com/competitions/optiver-trading-at-the-close/writeups/hyd-1st-place-solution>

9th place: <https://www.kaggle.com/competitions/optiver-trading-at-the-close/writeups/adam-9th-place-solution>

xth place: <https://github.com/liyian128/optiver-trading-at-the-close>

yth place: <https://fan2goal.github.io/mkdocs-material/blog/2023/12/24/kaggle-optiver-trading-at-the-close>

General option pricing: <https://github.com/Robin-Guillou/Option-Pricing>

general microstructure pricing: https://github.com/xhshenxin/Micro_Price